



AUTOPILOT HACKATHON — ROUND 2



THE FINALE

**Top 15. Mumbai.
One Night.**

Welcome to the Overnight **Finale.**

You've been chosen. Out of 300+ submissions across 15+ states, you are the Top 15 walking into Supervity's BKC office. Round 1 proved you could build an AI Intern. Round 2 is where you architect a fully autonomous workforce of AI Employees.



Trade Centre, BKC, Mumbai · May 16–18, 2026

FROM ROUND 1 TO MUMBAI

You Made It Through.

Round 1 was a 24-hour online sprint. The bar was high. Here's what came in:

300+

Submissions

From across India

15+

States Represented

Plus international entries

15

Top Teams Selected

Sales & Marketing tracks



AI Interns shipped: Lead Intelligence, CRM Update, Meeting Intelligence, Trend Analyser, Multi-Channel Content, Performance Ad Analyser, and more. You are sitting with the builders of those exact agents.

From AI Intern → AI Employees.

You are no longer building one specialist. You are architecting a governed, scalable software system — a Multi-Agent Ecosystem a Fortune 500 CFO would trust with real operations.

ROUND 1

A Single AI Intern

- One specialist Operator Agent
- Built entirely inside Supervity Auto
- 2+ external integrations
- Demo via video walkthrough

ROUND 2

A Workforce of AI Employees

- 4+ Operator Agents + 1 Orchestrator
- 1 mandatory Core Capability on Auto
- Custom Command Center UI from GitHub repo
- API-driven — agents triggered via Supervity APIs
- Live demo with exception handling on stage

ROUND 2 OBJECTIVES

What We Expect To See.

Six non-negotiables. Hit all six and you are in serious contention.

-  **4+ Operator Agents**
Each a domain specialist (CRM ops, comms, document handling, analytics, etc.).
-  **1 Orchestrator Agent**
The Manager that delegates, monitors, and passes stateful context to the Operators.
-  **1 Core Capability on Auto**
RAG, OCR, Voice, Web Search, Deep Research, or Pattern Mining — used by 2+ Operators.
-  **Custom Command Center UI**
Cloned from the starter repo, deployed locally, talking to your AI Employees via APIs.
-  **Dynamic AI Policies Enforced**
Plain-language guardrails the agents cannot violate. No hallucinations into production.
-  **Graceful Exception Handling**
When something breaks, route to the AI Workbench for Human-in-Command review.

THE ROUND 2 PROBLEM STATEMENT

Their bosses **just walked in.**

In Round 1 you fixed Om and Aarushi's individual workflows. In Round 2, their leadership wants a system that runs the whole department — governed, observable, and trustworthy.

TRACK 1 · SALES**Om's VP of Sales now needs the full pipeline.**

One agent updating a CRM isn't enough. She has 40 reps, an aging pipeline, leads slipping through unqualified, and zero confidence that policy is being followed before deals progress. She needs a Sales Command Center — a workforce of agents that capture intent from her website, qualify and route inbound leads, push approvals, generate proposals, and notify the right humans, with every decision audit-trailed.

TRACK 2 · MARKETING**Aarushi's CMO wants the whole growth engine.**

Cross-posting content is table stakes. The CMO needs a Marketing Command Center — agents that surface trends, draft on-brand copy, distribute, capture leads, audit campaigns, and pause the moment brand safety or budget rules are at risk. One System. One source of truth.

We tell you the destination. You **architect the route.**



THE DIRECTION

What we hand you

- A track (Sales or Marketing) and the persona's pain
- The minimum architecture — 4+ Operators + 1 Orchestrator
- The mandatory Core Capability menu (6 options)
- The four pillars to demonstrate live
- The starter repo, Auto API reference, and criteria



WHAT YOU ARCHITECT

This is where you win or lose

- Which 4+ Operators you build — your call
- Which Core Capability you build and how it's used
- Which policies matter to the business — your call
- How exceptions are routed and resolved
- What metrics belong on the Insights dashboard

THE 6TH NON-NEGOTIABLE

Build one **Core Capability** on Auto.

Prove the Auto platform can host advanced capabilities — not just business workflows. Pick one from the menu, build it as a callable agent, and wire it into the workforce.



Built on Auto

Not a Python library in your backend. The capability is its own agent on `auto.supervity.ai`, callable as an API.



Used by 2+ Operators or the Orchestrator

A capability nobody calls = no credit. It must do real work inside your AI Employees workforce.



Demoed live, end-to-end

Show a real call from an Operator triggering the capability, returning a result the workflow actually uses.



Why? Anyone can wire APIs together. This is how you prove Auto is a platform for **extensible AI** — and how you separate yourself from the field.

THE CORE CAPABILITY MENU

Pick One. Build It Well.

Six options. Different difficulty profiles. Judges will reward depth over breadth — build one capability that genuinely works.



RAG Pipeline

Ingest → chunk → embed → retrieve. Multi-file. Ground decisions in policy docs / SOPs.



OCR & Doc Parsing

Extract structured data from PDFs, invoices, contracts, screenshots. Handles messy uploads.



Web Search

Live grounding with source citations. Used by Trend Analyser / Lead Intelligence.



Deep Research

Multi-step planner-researcher. Decomposes a question, dispatches sub-queries, synthesises.



Automation Opportunity Miner

Reads execution logs, surfaces repeated exceptions, recommends new policies. Closes the loop.

 Pick based on your track. Sales → RAG for sales playbooks. Marketing → Web Search / Deep Research for trends. Either → Pattern Mining for the win.

THE ARCHITECTURE

How It All Connects.

Your Command Center (custom frontend) triggers your Orchestrator on Auto, which delegates to your AI Employees — 4+ Operators executing across external systems.



YOUR COMMAND CENTER (cloned from GitHub repo — runs locally via Docker)



triggers via Supervity API



ORCHESTRATOR AGENT · The Manager



**CRM
Ops**



**Comms
& Outreach**



**Data
Processing**



**Document
Mgmt**



**Core
Capability**

↓ *Executes across: SharePoint · Teams · HubSpot · Slack · GitHub · REST APIs · Excel*

The Four Pillars of Supervity Auto.

Judges will explicitly look for how you implement each of these.



AI Manager

The Control Center

Where your core logic lives. Build and orchestrate your multi-agent system. Define how the Orchestrator passes stateful context to Operators.



Dynamic AI Policies

The Guardrails

Plain-language business rules your agents cannot violate. Example: 'Never mark a deal Closed-Won without a signed contract attached.'



AI Workbench

Exception Management

When an API fails or data is missing, the agent must pause, route the work item here, and let a human resolve. This is THE most critical pillar.



AI Insights

Visibility & ROI

Live execution logs, reasoning traces, and ROI metrics. A CTO will only deploy AI they can monitor — show them what good looks like.

AI Policies in Action.

Plain-language guardrails enforced at runtime. Write them like an SOP. The Orchestrator checks every action against the active policy set before execution.

TRACK 1 · SALES

CRM Hygiene Policy

Trigger: Sales rep finishes a Zoom call.

Rule: Never mark a deal 'Closed Won' without (1) a signed contract uploaded to SharePoint AND (2) deal value confirmed by the rep.

Rule: If xyz fields are missing, do not push to CRM — route to Workbench.

Exception: On policy violation, agent pauses, notifies rep on Teams, waits for override.

TRACK 2 · MARKETING

Brand-Safe Publishing Policy

Trigger: Trend Analyser detects a viral topic on X.

Rule: Never publish content with competitor names, unverified claims, or pricing without marketing-lead approval.

Rule: All copy must pass brand-voice check AND clear the legal terms list before going to LinkedIn/X.

Exception: If brand-voice score < 0.8, route draft to Workbench for human edit.

AI Manager & AI Insights.

AI MANAGER — orchestration & stateful memory

Sales: Orchestrator receives a new inbound lead from HubSpot, delegates enrichment to **Lead Intelligence Agent**, passes the scored lead to **Outreach Agent** (with full context: company, intent signals, prior interactions), then asks **CRM Update Agent** to log the sequence — all in one stateful chain.

Marketing: Orchestrator pulls a trending topic from **Trend Analyser**, hands brand context + topic to **Multi-Channel Content Agent**, routes the variants to **Brand-Safety Agent**, and publishes — passing the campaign ID forward to ROI Optimizer the next morning.

AI INSIGHTS — live logs, reasoning traces & ROI metrics

Sales: Dashboard shows 247 leads enriched today, 12 routed to Workbench, avg. enrichment time 8.2s, \$1.4M pipeline touched — with one-click drill-down to the reasoning trace behind every decision.

Marketing: Live view of ad spend re-allocated in real time, top-performing creative variant, CPL trending down 18% — plus a 'why this decision?' modal that exposes the full agent chain.

Don't Build From Scratch.

We've shipped a production-grade boilerplate. Clone it, run it locally with Docker, and start wiring it to your Auto agents.



github.com/digitamizers/AutoPilot-Template



WHAT'S INCLUDED

- **FastAPI backend** with auto-generated Swagger docs
- **Next.js + React 19 frontend** with glassmorphic UI
- **PostgreSQL** + Alembic migrations
- **Pre-built pages:** Dashboard, Manager, Workbench, Settings
- **Audit logging** + role-based authz
- **Docker Compose** — one command to start



WHAT YOU BUILD

- **Connect AI Manager** chat to your agent orchestration backend
- **Build the policy engine** that evaluates rules at runtime
- **Build the insights engine** from your agent execution data
- **Wire up the Workbench** for exception routing
- **Trigger Supervity Auto agents** via the Auto API

GETTING STARTED

Up and Running in 6 Steps.

Prerequisites: Docker Desktop + Git. Windows users — enable WSL 2.

1 Clone

```
git clone <repo-url>  
cd AutoPilot-Template
```

2 Env file

```
cp .env.example .env  
(AUTH_BYPASS=true by default)
```

3 Start Docker

Open Docker Desktop
Wait for 'running' status

4 Boot services

```
macOS/Linux: make up  
Windows: docker compose up --build -  
d
```

5 Verify

```
docker compose ps  
→ 3 services 'running'
```

6 Open

App: localhost:3001
API Docs: localhost:8001/api/docs

From Operator Agent to Live Demo.

Five steps. Plan your architecture before you touch the keyboard.

- 01 Build Operators on Auto**
Use `auto.supervity.ai` chat-to-build to define each of your 4+ Operator Agents + your Core Capability. Set triggers, boundaries, exact output schemas.
- 02 Expose Them as APIs**
Every Auto agent gets a callable API endpoint. Copy the endpoints, secrets, and auth headers from `auto.supervity.ai/docs/reference/api-reference`.
- 03 Clone & Boot the Repo**
Spin up the starter (frontend + backend + DB) locally. The dashboard, AI Manager chat UI, and Workbench shell are already built.
- 04 Wire Frontend → Auto**
From the FastAPI backend, call your Auto agent APIs. The Orchestrator logic can live in your backend OR be its own Auto agent — your call.
- 05 Demo & Force an Exception**
Trigger the full workflow from your UI. Intentionally break one step. Show the Workbench catching it gracefully. This is how you win.

Use Any LLM. Any Stack. One Rule.

✅ YOU HAVE FULL FREEDOM ON

Policy engine — build your own runtime evaluator, or use an LLM-as-judge.

LLM choice — GPT, Claude, Gemini, Llama, Mistral, anything. Mix models per agent.

Frontend stack — extend the Next.js starter or swap in your own.

Vector DBs & tooling — Pinecone, Weaviate, pgvector, or whatever you pick (when RAG is your capability, the agent still lives on Auto).

Orchestrator implementation — in your backend, or as an Auto agent itself.

🔒 THIS MUST RUN ON SUPERVITY AUTO

Your 4+ Operators + Core Capability MUST be built on auto.supervity.ai using the chat-to-build interface.

Every agent MUST be callable as a Supervity API endpoint from your Command Center.

Why? *This is what judges will inspect live. If your agents are pure Python in your backend, you haven't built on the platform.*

JUDGING CRITERIA

How You Will Be Scored.

Execution, exception handling, and real-world value — not complexity for its own sake.

25%**Performance**

Reliable under enterprise conditions. When APIs fail, does it route to Workbench or crash?

25%**Business Value**

Would a CFO or Head of Sales deploy this Monday morning? Show quantifiable impact.

15%**Logical Reasoning & Architecture**

Is multi-agent orchestration defensible? Clean stateful context handoff?

15%**Platform Usage**

How effectively did you use AI Manager, Policies, Workbench, and Insights?

10%**Core Capability Depth**

Is the capability genuinely functional and integrated — or theater?

10%**Innovation & Execution**

Out-of-the-box thinking. Novel approaches to legacy bottlenecks.



THE OVERNIGHT TIMELINE

22 Hours. Plan Every One.

SATURDAY · MAY 16

11:30 AM Report in. Settle, meet your team.

12:00–1:00 PM Round 2 Workshop. Guidelines, Q&A.

1:00–2:00 PM Lunch + Planning

2:30 PM Build clock starts.

All night Build. FDEs on the floor. Snacks on tap.

SUNDAY · MAY 17

12:30 PM Code freeze. Step away from the keyboard.

1:00–2:00 PM Lunch. You made it through the night.

2:00–6:00 PM Judging. Live demo to F500 judges.

6:00–7:00 PM Closing ceremony. Winners announced.

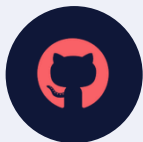
May 18 → Product Hunt launch. Top builds featured.

 *Build window: Sat 2:30 PM → Sun 12:30 PM = 22 hours. Architect before you code.*

YOUR REFERENCE TOOLKIT

Everything You Need. One Page.

Bookmark this slide. Come back to it at 3 AM.



Starter Repository

github.com/digitamizers/AutoPilot-Template

Clone first. Read /docs/command-center-guide.md.



Supervity Auto API Reference

auto.supervity.ai/docs/reference/api-reference

How to call your Operator Agents from your Command Center.



Build Your Agents Here

auto.supervity.ai

Chat-to-build interface. All 4+ Operators + your Capability live here.



Official WhatsApp Group

Source of truth • FDE office hours • Live unblocks

Drop your blockers here. FDEs are watching.

FINAL DIRECTIVE

Build the system a CFO would deploy Monday morning.



Robustness Over Complexity

A partial system that handles exceptions gracefully will always outscore an ambitious one that crashes silently.



Governance Over Ambition

Don't build for judges — build for the CTO who has to deploy this to 10,000 employees on Tuesday.

Let's build the future of Self-Driving Work. 🚀